

# Implementation Of Hybrid DNS Sinkhole Architecture Based On Pi-Hole And Machine Learning In Mitigation Of Online Gambling Site Access

Muhammad Rafa Ihsanuddin  
Yazzar  
*Prodi S1 Teknik Telekomunikasi  
Fakultas Teknik Elektro, Universitas  
Telkom  
Bandung, Indonesia  
rafaiy.2004@gmail.com*

Muhammad Damar Alkautsar  
Alfa Hakim  
*Prodi S1 Teknik Telekomunikasi  
Fakultas Teknik Elektro, Universitas  
Telkom  
Bandung, Indonesia  
damaralkautsar.724@gmail.com*

Muhammad Fachri Afif  
*Prodi S1 Teknik Telekomunikasi  
Fakultas Teknik Elektro, Universitas  
Telkom  
Bandung, Indonesia  
fachryafif86@gmail.com*

**Abstrak**— Online gambling in Indonesia has evolved into a systemic crisis, involving 12.3 million active players and a financial turnover of IDR 1,163.9 trillion between 2017 and 2025; meanwhile, syndicates exploit Domain Generation Algorithms (DGA) and hijack authoritative websites, causing static blacklist-based blocking methods to consistently lag behind. This research implements a Hybrid DNS Sinkhole architecture on a Raspberry Pi 4, featuring an inline proxy middleware based on a Random Forest model as the first line of defense and Pi-hole as a static filter for the second line, with all client queries forcibly redirected via Destination NAT on a MikroTik router. The model was trained on 5.1 million domains using 12 lexical features and TF-IDF weighting of 3–5 character N-grams, then calibrated at a threshold of 0.3559 through F3-Score optimization. Online testing against 600 active domains yielded a blocking rate of 98.00%, an allowing rate of 95.33%, a false negative rate of 2.00%, and a false positive rate of 4.67%, all meeting the targets of  $\geq 95\%$  and  $\leq 5\%$ . A Quality of Service evaluation based on TIPHON standards resulted in an index of 3.33 (Good) for daily usage—matching the baseline—with an average latency of 46.67 ms; under extreme load, the index dropped to 2.67 (Fair) due to computational bottlenecks in the edge device.

**Keywords**— DNS sinkhole, judi online, Pi-hole, *random forest*, *quality of service*

## I. INTRODUCTION

Online gambling in Indonesia has evolved from isolated legal infractions into a nationwide cyber and socio-economic crisis. A 2025 report by the Financial Transaction Reports and Analysis Center (PPATK) recorded 12.3 million active players, with a cumulative financial turnover of IDR 1,163.9 trillion between 2017 and 2025. Its reach extends to the most vulnerable groups, specifically 36,400 children under the age of 11 and 66,700 children aged 11–16 [1]. Consequent impacts include cycles of debt, entanglement with illegal online loans, and rising crime rates [1], [2].

Enforcement efforts to date have generally been reactive and reliant on blocking static domains, thereby creating what is termed an "illusion of eradication" [2]. Gambling syndicates possess high infrastructure resilience through Domain Generation Algorithms (DGAs) that generate

thousands of new domains daily; once a domain is blocked, operations instantly shift to an unregistered domain [3]. A more critical technical escalation involves the infiltration of high-reputation authoritative domains. By exploiting server vulnerabilities, attackers deface websites and inject malicious redirection scripts into legitimate sites—including government portals—turning them into "doorway pages" that lead to gambling servers [4], [5]. Administrators cannot block the parent domain without disrupting legitimate public services, resulting in a systemic blind spot in list-based filtering.

At the local network level, the most common mitigation solution is static DNS filtering, typically implemented using Pi-hole on single-board computers. This approach effectively blocks advertisements and harmful content, ensures privacy by keeping resolution within the local network, and introduces minimal latency [7], [12], [13]. Its fundamental weakness lies in the passive nature of its detection: the system only recognizes entries present on a blacklist. In contrast, cloud-based enterprise solutions have long utilized machine learning for proactive threat detection, enabling the identification of previously unseen variants [8], [9], [10], [11]—including DGA-generated domains—through lexical feature analysis of DNS traffic [6]. However, this approach requires routing all internal DNS queries to third-party providers, resulting in the logging of user browsing history and increased latency due to additional network routing [11], [18]. This research aims to mitigate this asymmetry by pursuing four objectives: developing a cost-effective and resource-efficient online gambling traffic filtering system; building a proactive detection model that recognizes DGA anomaly patterns; designing a mechanism to instantly block access to malicious DNS queries; and validating the classification accuracy and Quality of Service (QoS) feasibility of the resulting system. Its contributions include shifting proactive detection capabilities to low-cost single-board computers—with all decision-making occurring within the user's network [7], [15]—and prohibiting the whitelisting of authoritative domain suffixes to eliminate blind spots regarding subdomain hijacking.

## II. THEORITICAL REVIEW

### A. DNS Sinkhole and List-Based Filtering

A DNS sinkhole blocks access to malicious domains by returning a false response—typically the address 0.0.0.0 or an NXDOMAIN code—thereby preventing a connection to the destination server from ever being established. Detection is deterministic, resulting in minimal false positives, though its scope is limited to domains that have already been listed [12], [13].

### B. Domain Generation Algorithm and Authoritative Domain Exploitation

DGAs generate domain names algorithmically, producing character strings with distributions that deviate from organic word formation. This characteristic renders list-based approaches ineffective, as new domains are created faster than lists can be updated [3], [27]. In the context of online gambling in Indonesia, DGAs are combined with SEO poisoning—which hijacks the search rankings of high-authority sites—and the insertion of malicious redirects into compromised websites [4], [5], [14].

### C. Random Forest and Lexical Feature Extraction

Random Forest constructs multiple decision trees in parallel using bootstrap aggregating and determines the output based on a majority vote. Feature decorrelation at each split renders it resilient to the overfitting issues that plague single trees. Comparative studies consistently position Random Forest above other classical models in domain and URL classification: achieving 95.40% compared to Naive Bayes' 74.50% [19]; 96.20% versus Support Vector Machine's 95.10% (the latter struggling to map an optimal hyperplane in high-dimensional feature space) [20]; and 98.50% in an ensemble intrusion detection framework, contrasting with a single Decision Tree's 95.40% [17]. While Deep Neural Networks achieve slightly higher accuracy (97.40%), their matrix computational overhead is unsuitable for edge devices [20].

Lexical feature extraction maps the anatomy of domain character strings into a numerical vector space—encompassing length, Shannon entropy, digit ratio, vowel-to-consonant ratio, subdomain count, and Top-Level Domain risk weight—complemented by Term Frequency–Inverse Document Frequency (TF-IDF) weighting of character N-grams to capture transition patterns characteristic of gambling corpora and DGA-generated domains [6], [27], [28].

### D. Evaluasi Cost-Sensitive dan F3-Score

In security architecture, the penalty costs associated with different types of errors are asymmetric. Allowing a gambling domain to pass through results in losses that far outweigh the functional disruption caused by the erroneous blocking of a safe domain. Literature in other domains confirms similar asymmetries and rejects conventional accuracy metrics in favor of  $F\beta$  metrics that heavily weight recall [22], [23], while threshold tuning to balance false positives and false negatives has become standard practice in phishing detection models [29]. A value of  $\beta = 3$  makes recall approximately three times more critical than precision. Values of  $\beta \leq 2$  fail to provide sufficient penalty sensitivity,

whereas  $\beta \geq 4$  causes precision to collapse, as the model tends to block nearly all queries to avoid the penalty associated with allowing a malicious domain [22].

### E. Kualitas Layanan pada Server DNS

Service quality assessment relies on the TIPHON standard, which defines parameters for delay, packet loss, throughput, and jitter [16]. While the standard was formulated for flow-oriented multimedia traffic, DNS operates via brief, transactional request-response exchanges over UDP; consequently, three adjustments are required. Jitter is excluded because each DNS transaction is independent and concludes within a single packet exchange. Packet loss is redefined as "queries lost"—referring to queries sent but receiving no response within the timeout period. Throughput is expressed in queries per second (QPS) because DNS server performance is determined by computational capacity in processing queries rather than by the volume of data transferred [16]. As TIPHON does not specify quality categories for QPS, the threshold was derived from empirical measurements of a sinkhole server running on a dual-core processor with 4 GB of RAM; this setup recorded a peak rate of 24.48 QPS while remaining within TIPHON latency limits, a figure subsequently rounded to 25 QPS [21].

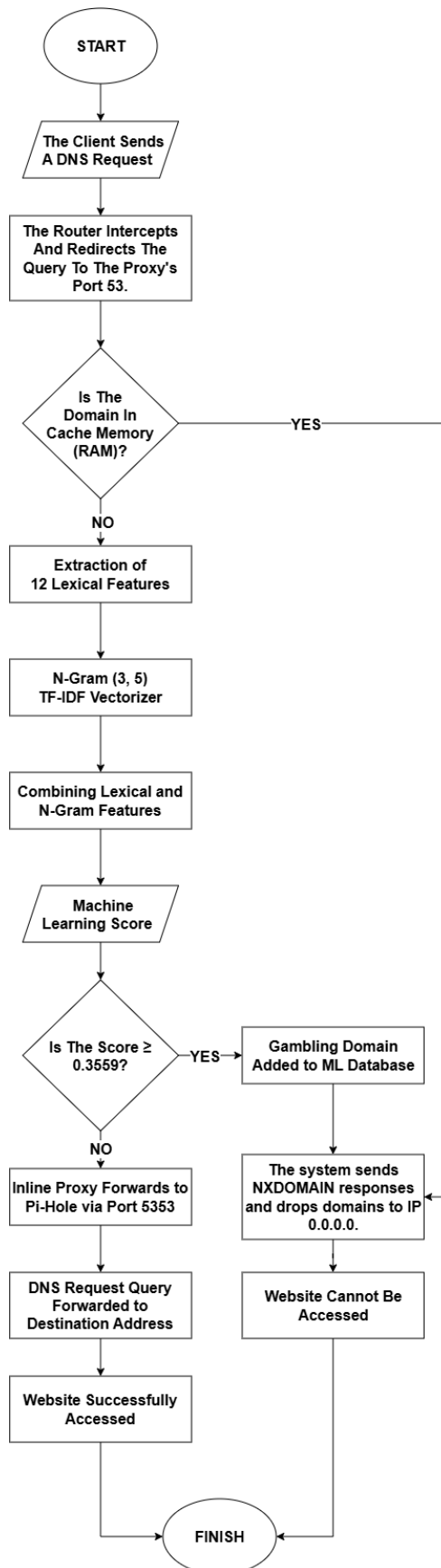
## III. METHODOLOGY

The research was conducted from February to August 2026, employing a design-and-build approach and empirical testing on an isolated local network. The total hardware budget was IDR 2,310,000.00, covering a Raspberry Pi 4 Model B (4 GB RAM), a MikroTik hAP Lite router, a modem, and network cables.

### A. System Architecture Design

The architecture utilizes a dual-path engine mechanism implemented on a single Raspberry Pi board. A MikroTik router is configured with Destination NAT rules to intercept all queries targeting port 53 (via UDP or TCP) and transparently redirect them to the local server, while excluding the server's own address to prevent routing loops. This mechanism eliminates the possibility of users bypassing DNS controls by configuring public resolvers on their devices and imposes minimal load on the router's processor, as the router only manipulates packet headers.

The complete query processing workflow is illustrated in Figure 1. The process begins when a client sends a DNS query, which the router intercepts and redirects to the proxy's port 53. At the front end, the `'inline_dns.py'` middleware script first checks whether the domain is already stored in the cache memory (RAM). If found, the system immediately returns an NXDOMAIN response and resolves the domain to the address 0.0.0.0 without performing re-inference, thereby preventing access to the site. If not found, the query enters the feature processing pipeline: extraction of 12 lexical features, N-gram (3, 5) transformation using a TF-IDF Vectorizer, merging of the two feature spaces, and execution of a Random Forest model to generate a machine learning score.



PICTURE 1

Hybrid DNS Sinkhole System Workflow Diagram

The score is compared against a threshold of 0.3559. If the score meets or exceeds this threshold, the domain is categorized as a gambling domain and added to the ML

database; the system then returns an NXDOMAIN response and redirects the domain to 0.0.0.0, preventing access to the site. If the score falls below the threshold, the domain is deemed safe, and the inline proxy forwards the query to the backend—specifically Pi-hole on port 5353—which acts as a static filter for ads, trackers, and malware. DNS queries that pass through both layers are then forwarded to the destination address, allowing successful access to the site.

## B. Justification for Model Selection

Conventional decision support system approaches, such as the Analytical Hierarchy Process, were not employed because they rely on pairwise comparison matrices manually populated by experts. Given a feature space of 10,012 dimensions—comprising 10,000 N-gram dimensions and 12 lexical dimensions—such an approach would require over 50 million pairwise evaluations and would be incapable of learning emerging zero-day domains [30], [24]. Machine learning was selected for its ability to perform stochastic generalization within sparse matrix spaces. Among the candidate algorithms, Random Forest was chosen as the core classification method based on three considerations: its resistance to overfitting through bagging, its robustness in selecting high-dimensional lexical features, and its low inference latency resulting from memory-level conditional logic execution.

## C. Dataset and Model Training

The training corpus consists of 2,613,730 gambling domains and approximately 2.5 million safe domains—totaling 5.1 million domains—split into 80% for training and 20% for offline testing. N-gram transformation employs 'TfidfVectorizer' with a 'char\_wb' analyzer, a character range of 3 to 5, sublinear scaling, and a feature retention limit of 10,000 to prevent model bloat. This matrix is combined with 12 lexical features: length, entropy, digit ratio, vowel ratio, hyphen ratio, subdomain count, TLD risk, consonant ratio, keyword matching, digit clusters, unique character ratio, and IP address patterns. Training is executed on a separate node (a personal computer), after which the model is serialized using 'joblib' and deployed to a Raspberry Pi; the local server thus performs only inference, adhering to the separation of training and serving phases common in MLOps practices [26].

## D. Test Scenario

Functionality testing was conducted on 300 gambling domains and 300 safe domains, all of which were active. Safe domain samples were manually collected via search engines across government, education, healthcare, e-commerce, and information portal categories, with their availability subsequently validated through a web browser. Gambling domain samples were gathered using Google Dorking techniques to capture three real-world attack vectors: SEO poisoning on high-authority domains, web defacement, and malicious redirection. The seamless access to authoritative domains observed during testing stemmed purely from the model's precision, not from whitelisting.

Service quality testing compared two scenarios. In the baseline scenario, the client connected directly to the router without Dst-NAT rules, allowing queries to be forwarded to the Internet Service Provider's resolver. In the sinkhole

scenario, Dst-NAT rules were enabled, forcing all queries to undergo inline inspection. Each scenario was evaluated under two load conditions: daily usage based on real activity from seven client devices recorded via Wireshark, and stress testing using dsnperf. Scalability was measured by monitoring processor utilization on the Raspberry Pi and MikroTik devices under loads ranging from one to seven simultaneous clients.

### E. Evaluation Parameters and Formulas

Mitigation effectiveness is measured using four ratios derived from the confusion matrix: blocking rate or True Positive Rate (TPR), allowing rate or True Negative Rate (TNR), False Positive Rate (FPR), and False Negative Rate (FNR), as shown in Equations (1) through (4).

$$\text{TPR} = \text{TP} / (\text{TP} + \text{FN}) \times 100\% \quad (1)$$

$$\text{TNR} = \text{TN} / (\text{TN} + \text{FP}) \times 100\% \quad (2)$$

$$\text{FPR} = \text{FP} / (\text{FP} + \text{TN}) \times 100\% \quad (3)$$

$$\text{FNR} = \text{FN} / (\text{FN} + \text{TP}) \times 100\% \quad (4)$$

...with TP representing successfully blocked gambling domains, TN representing safe domains allowed through, FP representing safe domains erroneously blocked, and FN representing gambling domains that slipped through. Model optimization was evaluated using the F $\beta$ -score with  $\beta = 3$ , as shown in Equation (5).

$$\text{F3} = (1 + \beta^2) \times (\text{P} \times \text{R}) / ((\beta^2 \times \text{P}) + \text{R}) \quad (5)$$

where P is precision and R is recall. Service quality parameters are calculated using Equations (6) through (8).

$$\text{Throughput} = \Sigma Q / T \quad (6)$$

$$\text{Latency} = (1/N) \Sigma (t_{\text{rest},i} - t_{\text{rest},i}) \quad (7)$$

$$\text{Queries Lost} = (\text{Q}_{\text{sent}} - \text{Q}_{\text{ans}}) / \text{Q}_{\text{sent}} \times 100\% \quad (8)$$

where  $\Sigma Q$  is the total number of recorded DNS packets, T is the recording time span, N is the number of answered queries, and  $\text{Q}_{\text{sent}}$  and  $\text{Q}_{\text{ans}}$  are the numbers of sent and answered queries, respectively. Since the units for these three metrics differ, each measured value is converted into a category score ranging from 1 to 4 and then averaged using Equation (9).

$$\text{Index QoS} = (\text{S}_{\text{thr}} + \text{S}_{\text{lat}} + \text{S}_{\text{ql}}) / 3 \quad (9)$$

Latency quality thresholds follow TIPHON standards: below 150 ms for index 4, 150–300 ms for index 3, 300–450 ms for index 2, and above 450 ms for index 1; the corresponding thresholds for lost queries are 0–2%, 3–14%, 15–24%, and above 24%. Throughput thresholds are set at above 25 QPS for index 4, 12.5–25 QPS for index 3, 6.25 to less than 12.5 QPS for index 2, and below 6.25 QPS for index 1, derived proportionally from the reference value of 24.48 QPS [21]. The performance specification targets to be met by the system are summarized in Table 1.

TABLE 1. SYSTEM PERFORMANCE SPECIFICATION TARGETS

| Testing Parameters  | Performance Targets   |
|---------------------|-----------------------|
| Blocking rate (TPR) | $\geq 95\%$ [27]      |
| Allowing rate (TNR) | $\geq 95\%$ [11]      |
| False positive rate | $\leq 5\%$ [11], [29] |
| False negative rate | $\leq 5\%$ [27], [29] |

The FPR and FNR limits are capped at 5% to ensure the classification algorithm does not block essential services, news portals, or legitimate government systems, while also ensuring no detection failures regarding lexical structures generated by DGAs.

## IV. RESULTS AND DISCUSSION

### A. Model Performance in Offline Testing

An offline evaluation was conducted on 995,322 domain samples, yielding the confusion matrix shown in the second column of Table 2. Substituting these values resulted in a precision of 99.96% and a recall of 99.99%, achieving an F3-Score of 0.9999. The absolute false negative count of 34—out of over half a million gambling domains—demonstrates the model's ability to counter the DGA camouflage techniques that typically bypass static databases. This result was obtained using a test distribution derived from the same training corpus; consequently, the online testing described in Subsection C serves as an external validation.

### B. Threshold Calibration and Feature Importance

The default probability threshold of 0.50 was lowered and fixed at 0.3559—the cutoff point yielding the highest F3-Score during the precision-recall curve iteration. Lowering the threshold shifts the decision boundary closer to the "safe" class distribution, thereby widening the detection net for the "gambling" class, consistent with the adopted cost-sensitive learning paradigm.

This value is not an arbitrarily set constant but rather a member of the set of unique probabilities actually output by the model. Random Forest calculates the probability of a domain x as the average class proportion across all tree estimators, as shown in Equation (10).

$$\text{P}(y = 1 | x) = (1 / N) \Sigma \text{P}_t(y = 1 | x) \quad (10)$$

where N is the number of decision trees—100 in this model—and  $\text{P}_t$  is the class probability at the terminal leaf of the t-th tree. The reconstruction of the 0.3559 value observed during the training phase is as follows: 35 trees placed the feature vector in a terminal leaf containing exclusively threat samples (resulting in  $\text{P}_t = 1.00$ ); one tree landed in a leaf containing 100 samples—comprising 59 threats and 41 safe domains—yielding  $\text{P}_t = 0.59$ ; and the remaining 64 trees projected the query to a leaf containing exclusively safe domains, resulting in  $\text{P}_t = 0.00$ . Substituting these three groups yields Equation (11).

$$\text{P} = (35 \times 1.00 + 1 \times 0.59 + 64 \times 0.00) / 100 = 0.3559 \quad (11)$$

All such unique values are collected into a set of threshold candidates  $T = \{0.0000, \dots, 0.3558, 0.3559, 0.3560, \dots, 1.0000\}$ , and each member is then evaluated using Equation (12)—which is the form of Equation (5) with  $\beta = 3$  substituted.

$$\text{F3}(\tau) = 10 \times [\text{P}(\tau) \times \text{R}(\tau)] / [9 \times \text{P}(\tau) + \text{R}(\tau)] \quad (12)$$

Substituting a precision of 0.9996 and a recall of 0.9999 at  $\tau = 0.3559$  yields an F3 score of 0.9999, the highest value across the entire set of candidates. This threshold was selected because shifting in either direction resulted in a decrease: at a stricter  $\tau$ , the gambling domain with a probability of exactly 0.3559 shifted to a False Negative, causing recall to plummet under the  $\beta = 3$  penalty; at a looser

$\tau$ , the system began blocking safe domains, causing precision to drop below the tolerance level. Since Random Forest is stochastic, this optimal point shifts proportionally with each retraining; consequently, the value of 0.3559 is specific to the model artifact used in this study, and reporting it requires including the training configuration to ensure reproducibility [31].

Feature importance analysis—commonly used to explain the decision-making basis of filtering models [25]—identifies ‘tld\_risk’ as the most influential variable, empirically reflecting the massive exploitation of low-cost Top-Level Domains by gambling infrastructures. The ‘keyword\_hit’ feature plays a critical role in detecting SEO poisoning injections: even when a domain utilizes an authoritative suffix resulting in a low ‘tld\_risk’ score, the inclusion of gambling-related terms in the subdomain triggers a score spike, necessitating a block. This mechanism enables the enforcement of strict blocking rules without compromising access to authoritative domains. The ‘subdomain\_count’ feature exposes subdomain hijacking techniques, while entropy, vowel/consonant/digit ratios, and digit clusters measure the randomness characteristic of DGA-generated domains. Conversely, ‘has\_ip\_pattern’ yields zero impurity reduction because DNS protocols process only Fully Qualified Domain Names, causing this feature to be naturally eliminated.

### C. Online Testing of 600 Active Domains

A summary of the confusion matrix and its derived ratios for both testing stages is presented in Table 2.

TABLE 2. CONFUSION MATRIX AND TEST RESULT RATIOS

| Parameters          | Offline Testing (995,322 samples) | Online Testing (600 active domains) |
|---------------------|-----------------------------------|-------------------------------------|
| True Positive (TP)  | 522,712                           | 294                                 |
| True Negative (TN)  | 472,383                           | 286                                 |
| False Positive (FP) | 193                               | 14                                  |
| False Negative (FN) | 34                                | 6                                   |
| Blocking rate (TPR) | 99,99%                            | 98,00%                              |
| Allowing rate (TNR) | 99,96%                            | 95,33%                              |
| False positive rate | 0,04%                             | 4,67%                               |
| False negative rate | 0,01%                             | 2,00%                               |

In testing against 300 active gambling domains, the system blocked 294 domains, achieving a blocking rate of 98.00%. The six instances of evasion revealed three failure patterns: lexical dilution, where an anomalous substring is masked by a dominant safe lexicon, causing N-gram feature weights to drop and the score to remain below the threshold; high-level lexical camouflage involving normal terminology devoid of explicit gambling keywords—appearing in web defacements of legitimate, low-entropy domains; and out-of-vocabulary cases, involving unusual slang terms not represented in the training corpus.

In testing against 300 safe domains, the system permitted 286 domains, achieving an allowance rate of 95.33%. All false-positive cases stemmed from lexical overlap between legitimate domains and the gambling corpus. Some were triggered by partial keyword matches—substrings embedded

within legitimate entity names or promotional words common to gambling domains—while others resulted from structural anomalies, such as excessively short root domains distorting vowel-to-unique-character ratio normalization, complex subdomain structures resembling hijacking patterns, and the use of high-risk TLDs. Such errors can be mitigated administratively via absolute whitelisting on the backend Pi-hole interface, thereby avoiding the need to relax thresholds at the frontend.

The discrepancy between the two columns in Table 2 confirms that offline metrics reflect the system's discriminative capability regarding corpus distribution, whereas online metrics reflect performance on real-world traffic containing previously unrepresented camouflage vectors. Nevertheless, all four security parameters met the specification targets outlined in Table 1.

### D. Network Service Quality (QoS)

The service quality measurement results for both scenarios and both load conditions are summarized in Table 3.

TABLE 3. PERFORMANCE COMPARISON OF BASELINE AND HYBRID DNS SINKHOLES

| Parameters      | Daily Base | Stress Base | Daily Sinkhole | Stress Sinkhole |
|-----------------|------------|-------------|----------------|-----------------|
| Minimum Latency | 1,76 ms    | 1,96 ms     | 3,82 ms        | 2,02 ms         |
| Maximum Latency | 1294,87 ms | 4241,41 ms  | 991,47 ms      | 4804,12 ms      |
| Average Latency | 66,38 ms   | 348,75 ms   | 46,67 ms       | 416,39 ms       |
| Throughput      | 6,52 QPS   | 29,75 QPS   | 7,64 QPS       | 48,69 QPS       |
| Queries lost    | 0%         | 0,83%       | 1,38%          | 23,22%          |
| Index QoS       | 3,33       | 3,33        | 3,33           | 2,67            |

Under daily usage conditions, the sinkhole architecture achieved a QoS index of 3.33 (categorized as "Good"), matching the baseline performance observed prior to the system's implementation. This result reflects an aggregate of metrics: an average latency of 46.67 ms (index 4), a throughput of 7.64 QPS (index 2), and a query loss rate of 1.38% (index 4). Latency in the sinkhole scenario was actually lower than the baseline because fast-path caching bypassed the resolution path for recurring queries, meaning most daily queries did not trigger model inference. It should be emphasized that the daily throughput represents the actual request rate from seven clients during the observation period—not the server's capacity limit; the low QPS value indicates a low demand load.

Under extreme load, the QoS index dropped to 2.67 (categorized as "Moderate"), falling below the baseline, which remained at 3.33. Average latency surged to 416.39 ms and query loss spiked to 23.22% (both rated index 2). The cause was a hardware bottleneck in the edge device's ARM processor: when flooded with a massive volume of queries for domains not present in the cache, the processor was fully utilized for feature extraction and on-the-fly inference, causing socket buffer queues to build up and forcing the system to drop some queries.

Interestingly, throughput moved in the opposite direction, reaching 48.69 QPS—surpassing the baseline peak of 29.75

QPS and earning an index of 4. These two parameters must be interpreted in tandem: the server processed incoming queries at the maximum possible rate, yet not all transmitted queries were successfully serviced. This combination resulted in a final index falling into the "Moderate" category. It should be noted that stress tests across scenarios were not entirely equivalent due to differing injection volumes; therefore, the figures are best interpreted as indicators of the device's saturation point.

#### E. Scalability Based on Processor Utilization

Measurements of computational load for 1 to 7 simultaneous client devices are presented in Table 4.

TABLE 4. PROCESSOR UTILIZATION VS. NUMBER OF CLIENTS

| Number of Clients | CPU Raspberry Pi (%) | CPU MikroTik (%) |
|-------------------|----------------------|------------------|
| 1                 | 14,88                | 2,4              |
| 2                 | 37,29                | 3,3              |
| 3                 | 43,11                | 3,9              |
| 4                 | 46,52                | 4,9              |
| 5                 | 44,88                | 5,9              |
| 6                 | 41,05                | 6,4              |
| 7                 | 46,85                | 8,2              |

Router processor utilization increased linearly and in a controlled manner from 2.4% to 8.2%, demonstrating the high efficiency of the Dst-NAT mechanism, as the router merely manipulates packet headers. On the server side, a surge from 14.88% to 37.29% occurred when the load increased from one to two clients, indicating overhead associated with script initialization and background services. However, within the range of 3 to 7 clients, utilization stabilized between 41% and 46% without reaching saturation. This stability confirms the effectiveness of fast-path caching: as the number of clients increases, the likelihood of encountering previously processed queries rises, allowing them to be resolved from the cache without re-inference. These findings demonstrate that performing whitelist checks on authoritative domains remains manageable for low-power devices.

#### F. Compliance with Specifications and Development Direction

All security parameters were met: the blocking rate (98.00%) and allowing rate (95.33%) exceeded the 95% minimum target, while the FPR (4.67%) and FNR (2.00%) remained below the 5% tolerance threshold. Daily service quality and processor scalability requirements were also satisfied. The only specification not met was service quality under extreme load—a trade-off consciously accepted to maintain local control, privacy, and cost-efficiency.

Four development paths may be pursued. First, periodic recalibration of the keyword dictionary to improve sensitivity to lexical dilution and out-of-vocabulary cases. Second, implementation of manual, absolute whitelisting as a backend measure to compensate for false positives. Third, upgrading server processor specifications to reduce bottlenecks during N-gram matrix extraction. Fourth, integrating decryption and

filtering capabilities for encrypted DNS protocols—such as DNS over HTTPS and DNS over TLS—given that the current system is limited to inspecting conventional queries on port 53. Further testing using standardized injection volumes across scenarios is also recommended.

## V. CONCLUSION

A hybrid DNS sinkhole architecture—combining a Random Forest-based inline middleware at the front end with a static Pi-hole filter at the back end—has proven capable of proactively mitigating access to online gambling sites using low-cost single-board computers. A model trained on 5.1 million domains, utilizing a fusion of 12 lexical features and TF-IDF N-gram weighting, achieved an F3-Score of 0.9999 in offline testing; meanwhile, online testing against 600 active domains yielded a blocking rate of 98.00% and an allowing rate of 95.33%, both exceeding the 95% minimum target. Calibrating the threshold to 0.3559 via a cost-sensitive learning paradigm reduced the false negative rate to 2.00% while maintaining the false positive rate at 4.67%—below the 5% tolerance limit—thereby enabling the enforcement of authoritative domain whitelisting without blocking access to legitimate public services. Regarding service quality, the insertion of the inference layer did not degrade daily browsing; the QoS index remained at 3.33 with an average latency of 46.67 ms (matching the baseline), although the index dropped to 2.67 under extreme loads due to computational bottlenecks in the edge device rather than compromised transmission integrity. Fast-path caching proved crucial in maintaining stable processor utilization—ranging from 41% to 46%—while serving seven simultaneous clients. Consequently, proactive detection capabilities previously accessible only at the corporate level can now be deployed in small-scale networks without licensing costs or the need to surrender user browsing records to external entities.

## REFERENCE

- [1] Pusat Pelaporan dan Analisis Transaksi Keuangan (PPATK), "Laporan Analisis Strategis Transaksi Keuangan Mencurigakan untuk Mencegah Dampak Sosial Judi Online di Indonesia," PPATK, 2025.
- [2] A. P. Satria, "Judi Online dan Ilusi Pemberantasannya," *JIMU: Jurnal Ilmiah Multidisipliner*, vol. 3, no. 4, pp. 2245–2252, Jul. 2025.
- [3] F. Mashood and M. Nabeel, "A Longitudinal Study of Recently Observed Malicious Domains: Characteristics, Infrastructure, and Abuse Patterns," *arXiv preprint arXiv:2606.11111*, Jun. 2026.
- [4] E. S. Alim et al., "Monitoring dan Pencegahan Serangan Judi Online (Slot Gacor) pada Website," *EDUMATIC*, vol. 8, no. 1, pp. 75–83, Jun. 2024.
- [5] H. Wang et al., "Real-PGDN: A Two-level Classification Method for Full-Process Recognition of Newly Registered Pornographic and Gambling Domain Names," *arXiv preprint arXiv:2511.22215*, 2025.
- [6] L. Torrealba Aravena et al., "Malicious Domain Names Detection with DeepDGA, a Hybrid Character and Word Embeddings Deep Learning Architecture," in *Proc. 21st Int. Conf. Netw. Serv. Manage. (CNSM)*, 2025, pp. 1–8.
- [7] D. I. Mulyana et al., "Network Security Optimization Against Online Gambling and Pornography Sites Using DNS Filtering

- and Orangeipi," *MALCOM*, vol. 4, no. 2, pp. 647–655, Apr. 2024.
- [8] S. L. Kuna et al., "Malicious Domain Detection Using Integrated Supervised and Unsupervised Machine Learning Approaches," *Journal of Theoretical and Applied Information Technology*, vol. 103, no. 13, pp. 4744–4752, Jul. 2025.
- [9] M. Hasibuan et al., "Classification of Phishing URL Attacks Using Random Forest Algorithm Based on Feature Importance," *Bit-Tech*, vol. 8, no. 2, pp. 2985–2993, 2025.
- [10] U. S. D. R. A. Patil, and Mohana, "Malicious URL Detection and Classification Analysis using Machine Learning Models," in *2023 Int. Conf. on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)*, 2023, pp. 470–476.
- [11] S. Sallam, M. El Barachi, and N. Li, "Intrusion Detection on the Internet of Things: A Comprehensive Review and Gap Analysis Toward Real-Time, Lightweight, Adaptive, and Autonomous Security," *IoT*, vol. 7, no. 1, pp. 16–38, Feb. 2026.
- [12] M. Rahman, "Implementasi Web Content Filtering Pada Jaringan RT/RW Net Menggunakan Pi-Hole DNS Server," *Generation Journal*, vol. 7, no. 1, pp. 50–55, Mar. 2023.
- [13] Nurasmawati JR, "Penerapan Pi-Hole DNS Server sebagai Ad-Blocker dan Sistem Filtering Website pada Jaringan Komputer," Skripsi, Univ. Negeri Makassar, 2023.
- [14] H. Pridana, "Mapping the Threat of Misuse of Blockchain Domain Name System (BDNS) Technology in Online Gambling in Indonesia," *Security Intelligence Terrorism Journal*, vol. 2, no. 2, pp. 229–235, 2025.
- [15] E. D. Sitanggang et al., "Pengembangan Layanan Pemblokiran Situs Bermuatan Negatif menggunakan DNS Sinkhole dan Layanan DNS Quad 9 dengan Metode PPDIOO," *LOFIAN: Jurnal Teknologi Informasi dan Komunikasi*, vol. 3, no. 2, pp. 16–24, Feb. 2024.
- [16] I. S. Nisa, R. M. Saputro, T. F. Nugroho, and A. R. Lahitani, "Analisis Quality of Service (QoS) Menggunakan Standar Parameter Tiphon pada Jaringan Internet Berbasis Wi-Fi Kampus 1 Unjaya," *Teknomatika: Jurnal Informatika dan Komputer*, vol. 17, no. 1, pp. 1–9, Apr. 2024.
- [17] S. Singh, A. Charak, S. Dutta, and J. Mahajan, "Ensemble Learning-Based Intrusion Detection and Automatic Mitigation Framework for DDoS Attacks in SDN Environments," *Int. J. Eng. Res. Technol. (IJERT)*, vol. 15, no. 6, Jun. 2026.
- [18] A. M. Syabriansyah and A. Sunyoto, "Optimasi Deteksi Situs Judi Online Menggunakan Algoritma Decision Tree untuk Mitigasi Risiko Kebocoran Data Pribadi," *DECODE: Jurnal Pendidikan Teknologi Informasi*, vol. 6, no. 2, pp. 321–333, Jul. 2026.
- [19] A. Jadhav and P. Chandre, "A Hybrid Heuristic-Machine Learning Framework for Phishing Detection Using Multi-Domain Feature Analysis," *Eng. Technol. Appl. Sci. Res.*, vol. 15, no. 5, pp. 27219–27226, 2025.
- [20] S. V. Patil and V. P. Singh, "An Adaptive Phishing Detection System Based on Heuristic Algorithms, NLP Analysis, and Canopy-Optimized Feature Selection," *Int. J. Drug Deliv. Technol.*, vol. 16, no. 58s, pp. 1627–1641, Jun. 2026.
- [21] G. Ohak, "Enhancing Small Business Cybersecurity with Multi-Layer Threat Blocking and Retrospective Hunting," Master's thesis, Tallinn Univ. of Technology, Tallinn, Estonia, 2026.
- [22] Y.-L. Lee, D.-C. Juan, X.-A. Tseng, Y.-T. Chen, and S.-C. Chang, "DC-Prophet: Predicting Catastrophic Machine Failures in DataCenters," *arXiv preprint arXiv:1709.06537*, Aug. 2017.
- [23] J. Denker, L. Al-Shrouf, and M. Jelali, "Multiclass Defect Classifier from Legacy Foundry Data: A Decision Support System for Manual Inspection Time Reduction," *Preprints.org*, Jul. 2026, doi: 10.20944/preprints202607.1269.v1.
- [24] L. Vaccari, E. Balugani, F. Lolli, and R. Gamberini, "A Machine Learning and Multi-Criteria Decision-Making Approach to Cycle Counting," *Logistics*, vol. 10, no. 1, p. 10, Dec. 2025, doi: 10.3390/logistics10010010.
- [25] S. S. Shafin, "An explainable feature selection framework for web phishing detection with machine learning," *Data Sci. Manag.*, vol. 8, pp. 127–136, 2025.
- [26] D. Kreuzberger, N. Kühn, and S. Hirschl, "Machine Learning Operations (MLOps): Overview, Definition, and Architecture," *IEEE Access*, vol. 11, pp. 31866–31879, 2023, doi: 10.1109/ACCESS.2023.3262171.
- [27] S. M. Aljabri et al., "Detection of Malicious URLs Using Machine Learning Techniques: A Review," *IEEE Access*, vol. 11, pp. 86479–86498, 2023, doi: 10.1109/ACCESS.2023.3303867.
- [28] M. R. N. M. B. Amin et al., "A Comprehensive Review of Machine Learning and Deep Learning-Based Solutions for IoT Cyber Security," *IEEE Access*, vol. 11, 2023.
- [29] S. Dalvi, G. Gressel, and K. Achuthan, "Tuning the False Positive Rate / False Negative Rate with Phishing Detection Models," *International Journal of Engineering and Advanced Technology (IJEAT)*, vol. 9, no. 1S5, 2019.
- [30] M. A. Alves, I. R. Meneghini, A. Gaspar-Cunha, and F. G. Guimarães, "Machine Learning-Driven Approach for Large Scale Decision Making with the Analytic Hierarchy Process," *Mathematics*, vol. 11, no. 3, p. 627, Jan. 2023, doi: 10.3390/math11030627.
- [31] J. Pineau et al., "Improving Reproducibility in Machine Learning Research (A Report from the NeurIPS 2019 Reproducibility Program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.